

优秀系统设计与工程保证方法论

从约束、不变量与认识边界，推导可运营、可演进、可验证的工程系统

ESD-METHOD-001

v2.0.0-draft.1

DRAFT / NORMATIVE FOUNDATION

权威内容来源为 Markdown。本件仅供阅读与评审，未获正式发布批准。

阅读与版本说明

PREVIEW / DRAFT · 非权威阅读视图

本件保留源文档的标题、条款编号、版本与草案状态，不构成新的规范权威。封面、导航、索引与排版标签为生成内容，不能代替正文中的适用范围、条件或批准记录。

制品标识 ESD-METHOD-001

版本 2.0.0-draft.1

状态 DRAFT / NORMATIVE FOUNDATION

阅读密度 balanced · Visual Profile Candidate，尚未冻结

基准 commit 783c7d3b9b6a1e4f715a960c18e687604143f98b

准备标识 353090fade94323347038eee

使用方式 目录与书签可跳转；合订版中的核心文档链接跳转到本件对应篇章。独立版在源字节匹配时提供固定 commit 链接，否则指向随附的 source 快照。完整输入以准备记录为准，可能包含未提交变更；正文原有编号不重新连续编号。

ESD-METHOD-001 v2.0.0-draft.1 DRAFT / NORMATIVE FOUNDATION

目录

阅读与版本说明	2
优秀系统设计与工程保证方法论	4
1. 文档定位	5
2. 严格定义：什么是优秀的系统	6
3. 认识论边界：工程保证能够证明什么	6
4. 系统设计十条总纲	8
5. 完整的架构推导模型	10
6. 从问题、约束与风险开始	11
7. 关键性质：从愿望到带作用域的契约	13
8. 状态、身份、边界与所有权	15
9. 失败、一致性、提交与恢复	17
10. 有界资源、背压与动态稳定性	19
11. 容量、单位经济性与复杂度预算	21
12. 契约、安全、演进与可逆性	23
13. 工程保证：从设计主张到持续证据	24
14. 组织、成熟度与渐进式采用	27
15. 最终判定标准	28
附录 A：核心术语边界	29
附录 B：参考资料	29
附录 C：历史 v1.1.0 关键校准	30

优秀系统设计与工程保证方法论

从约束、不变量与认识边界，推导可运营、可演进、可验证的工程系统

文档属性	内容
文档编号	ESD-METHOD-001
版本	2.0.0-draft.1 (基于 1.1.0)
状态	DRAFT / NORMATIVE FOUNDATION; 未获发布批准
文档角色	定义系统设计与工程保证的原则、术语、推导顺序和认识边界
配套文档	ESD-REFERENCE-001 《优秀系统设计：从约束、不变量到证据》
适用对象	架构师、Staff/Principal Engineer、技术负责人、平台与基础设施工程师
适用系统	分布式服务、数据平台、流处理、存储系统、控制面、高性能系统与关键业务系统
权威格式	本 Markdown 文件；任何 DOCX、PDF 或网页均只是非权威视图

文档关系与冲突规则

本文对通用设计原则、核心概念与认识边界拥有主题权威；不是覆盖所有领域的最高优先级文档。主题划分以 [ESD-SUITE-000 §3.1](#) 为准：治理、风险与 Gate 状态由 GOV 定义；Claim、Assumption 与 Evidence 状态由 ASSURANCE 定义；运行与恢复契约由 OPS 定义；REFERENCE 提供执行流程与示例。ADR 记录决策及其依据，不自行授予豁免权；偏离要求必须经过 GOV 规定的授权程序。

摘要

优秀的系统不是技术组件的集合，也不是一张在正常路径上看起来完整的架构图。它应当是在明确的业务、技术、组织、经济、安全与时间约束下，以尽可能低的必要复杂度持续维护关键性质；在失败、过载、升级、依赖退化和需求变化中保持可接受行为；并通过与具体制品、配置、环境和工作负载绑定的证据，说明这些性质在既定覆盖范围内确实成立。

这一定义包含三个彼此不可替代的层次：

- 架构推导**：由问题、约束、工作负载、风险和不变量推导状态、边界、语义与机制；
- 工程控制**：通过所有权、有界资源、准入、背压、恢复、版本化和变更控制维持系统稳定；
- 工程保证**：把设计主张转换为已编码属性、验证机制、证据、门禁和运行反馈，同时诚实保留无法证明的 UNKNOWN。

本文反对两种对称的错误：一端是由时髦技术名词驱动的组件堆砌；另一端是以敏捷为名拒绝处理高代价、难逆转决策。正确方法不是“先设计一切”，而是：

对高后果、高不确定性和难逆转的决策提前建模；对低后果、可逆的实现细节延迟决定，并用原型、基准和运行证据收敛。

1. 文档定位

1.1 目标

本文建立一套可长期复用的系统工程认知基线，用于回答以下问题：

- 系统为什么存在，哪些结果才具有业务价值；
- 哪些性质在任何允许场景下都不能被破坏；
- 哪些状态是权威事实，由谁拥有、修改、恢复和运营；
- 网络、进程、依赖、配置和资源失败时系统如何收敛；
- 一致性、交付、幂等、恢复和降级语义如何从不变量推导；
- 吞吐、时延、容量和单位成本如何计算；
- 发布、回滚、重放、迁移、应急和事故学习如何进入架构；
- 关键主张如何被验证，证据能证明什么、不能证明什么；
- 当假设失效、环境漂移或证据过期时，系统如何降低确定性而不是伪造 PASS。

1.2 非目标

本文不是：

- 某种语言、数据库、消息系统、云平台或 Kubernetes 的选型目录；
- 对微服务、事件驱动、CQRS、Service Mesh 或 Event Sourcing 的默认推荐；
- 要求所有项目在编码前完成同等厚度的“大设计”；
- 以文档数量、门禁数量或测试覆盖率替代真实风险降低；
- 对未编码属性、未知环境或开放世界行为提供绝对正确性承诺。

1.3 规范性用语

- **必须 (MUST)**：缺失会直接威胁关键正确性、安全性、可恢复性或生产可控性；
- **应 (SHOULD)**：默认要求，偏离时必须记录理由、风险、Owner 和复审条件；
- **可以 (MAY)**：可根据风险、成本与团队能力选择；
- **不得 (MUST NOT)**：会形成不可接受风险或破坏本框架的认识边界；
- **未知 (UNKNOWN)**：当前证据不足，不能合理判定为通过或失败；
- **不适用 (NOT_APPLICABLE)**：经论证后该要求与当前系统或变更无关。

2. 严格定义：什么是优秀的系统

优秀的系统，是一个受约束的社会技术控制系统：它以尽可能低的必要复杂度，在明确的状态、身份、责任与资源边界内持续维护关键性质；能够在正常、降级、恢复和紧急模式之间安全转换；并通过有作用域、有来源、有时效的证据说明这些性质在当前制品、配置、环境、工作负载和故障模型下成立。

这个定义刻意包含以下限定：

- **受约束**：架构不是技术自由发挥，而是业务效用、风险、成本和工程能力共同约束下的选择；
- **社会技术系统**：代码、基础设施、数据、配置、权限、团队、流程和应急主体共同构成系统；
- **关键性质**：正确性必须表达为可检查的安全、活性、持久性、安全隔离和恢复要求；
- **运行模式**：系统不只有“正常/失败”，还存在受控降级、隔离、恢复、维护 and 紧急模式；
- **必要复杂度**：复杂度必须为明确价值或风险控制承担举证责任；
- **证据作用域**：任何验证结论都只对其覆盖的制品、配置、环境、输入分布和故障场景有效。

架构因此不是：

Kafka + Kubernetes + Redis + Rust + Service Mesh

也不是：

一张包含所有服务和箭头的拓扑图

架构更接近：

一组关于状态、身份、边界、失败、资源、控制、演进和证据的版本化决策，以及用于持续维护这些决策所声明性质的机制。

3. 认识论边界：工程保证能够证明什么

3.1 验证工具不是绝对仲裁者

编译器、类型系统、静态分析、模型检查、测试套件、策略引擎、CI 和运行时监控，只能裁决它们已经表达、编码和覆盖的属性。它们不能自动发现所有隐含业务假设，也不能从有限输入空间推出开放世界中的绝对正确性。

因此，高质量工程语言应避免：

测试通过，所以系统正确。

结论必须与验证方法匹配：

方法	可以支持的结论	不能自动推出的结论
有限测试、故障注入	在已执行的输入与轨迹中，断言未被违反	所有输入与调度均正确
模型检查	在明确模型、假设与已探索状态空间内性质成立；未完成探索必须声明边界	实现与开放环境自动符合模型
定理证明	在形式化定义、前提及证明系统内命题成立	前提在生产恒真或代码自动满足规格
运行监测	在信号覆盖、采样与检测能力内得到运行事实	未报警即不存在违反

Safety 通常可由有限坏前缀反驳；有界活性可由截止时间违反反驳；无界活性不能仅凭有限时间“尚未完成”否定，需公平性、依赖和资源假设以及进展论证；SLO 需统计窗口、样本与测量不确定性。可裁决不等于所有性质都能被有限运行观测直接证伪。具体义务见 [ASSURANCE §5.2](#)。

这不是削弱工程保证，而是避免贬低形式证明，也避免把模型内证明夸大为实现或开放世界的保证。

3.2 四层主张模型

每个重要设计结论应区分四个层次：

层次	含义	典型错误
Desired Claim	业务或架构希望成立的性质	把愿望直接写成事实
Encoded Property	已被契约、类型、策略、断言或测试形式化的性质	未编码却声称已覆盖
Verified Result	验证器在给定范围内得到的结果	忽略输入、环境与工具边界
Evidence-backed Decision	与当前制品和环境绑定、足以支持门禁的证据	证据过期、错绑或不可复现

只有完成从主张到证据的映射，系统才具备工程保证意义上的可裁决性。

3.3 UNKNOWN 是一等状态

以下情况不得被解释为 PASS：

- 没有运行验证；
- 验证工具不覆盖该性质；
- 证据无法关联当前制品或配置；
- 环境、依赖或工作负载已经超出验证范围；
- 证据过期、损坏或不可复现；
- 关键假设没有被验证或监测。

此时应返回：

UNKNOWN / INSUFFICIENT_EVIDENCE

成熟系统不是永远绿色，而是能够准确暴露当前不知道什么。

3.4 证据不是永久有效

任何下列变化都可能使既有证据失效：

- 二进制、依赖、编译器或构建参数变化；
- 配置、策略、Schema、DBC、模型或参考数据变化；
- 节点规格、内核、网络、存储后端或区域变化；
- 输入分布、租户偏斜、对象大小或突发模式变化；
- 外部依赖语义或 SLO 变化；
- 新事故暴露了未建模故障路径。

因此，证据必须具有主体身份、环境身份、覆盖范围、产生时间和失效规则。

4. 系统设计十条总纲

总纲一：约束先于架构

系统设计必须先明确业务、技术、组织、经济、安全、合规和时间约束，再讨论组件与部署形态。团队技能、既有运维能力和迁移窗口本身就是硬约束，不得在选型后才补写理由。

最低实践：建立版本化的 Constraint / Assumption Register。

总纲二：关键性质先于机制

需求描述“希望发生什么”，关键性质描述“无论发生什么都不能被破坏什么”。事务、消息队列、锁、租约、幂等键和副本只在维护明确性质时才有正当性。

最低实践：为每项高后果风险建立带作用域的 Critical Property Contract。

总纲三：状态权威与责任必须显式

每份持久状态必须有唯一权威来源；每个关键组件同时必须有技术 Owner、运营 Owner 和必要时的安全 Owner。共享可读不等于共享可写，复制不等于多权威。

最低实践：维护 State Ownership Register，明确写权限、恢复来源和冲突裁决。

总纲四：失败、过载与恢复是一等设计输入

网络超时、进程崩溃、重复、乱序、部分成功、磁盘写满、依赖变慢和配置错误不是异常边角，而是生产系统的正常状态空间。系统必须事先定义失败后的可观察行为、恢复权威、最大影响和人工介入点。

最低实践：维护 Failure / Recovery Matrix 与显式运行模式。

总纲五：所有可增长资源必须有界

线程、任务、队列、连接、缓存、inflight bytes、重试次数、重试时间、临时文件和待补偿工作都必须有上界。无界异步不是吞吐设计，而是把压力隐藏到 OOM 或级联故障发生。

最低实践：每个队列和缓冲区必须拥有容量单位、满载行为、背压方向、Owner 与观测指标。

总纲六：一致性应选择最弱但足够的语义

系统既不应默认强一致，也不应把最终一致当作分布式系统信仰。应从业务不变量反推需要的原子性、顺序、隔离、可见性和冲突语义。at-least-once + idempotent commit 只有在稳定操作身份和原子条件写明确时，才形成 exactly-once business effect。

最低实践：对每个外部副作用明确提交点、ACK 顺序、幂等身份和冲突结果。

总纲七：控制面、数据面、管理面与保证面分离

高吞吐数据路径不应被频繁变化的策略污染；策略不应与执行机制混为一体；管理操作不应绕过正常控制；验证规则和门禁本身也必须受保护和审计。

最低实践：明确四个平面的状态、权限、接口和失效模式。

总纲八：容量、成本和动态稳定性必须可计算

平均吞吐不足以证明系统稳定。设计必须覆盖峰值、恢复积压、依赖降容、重试放大、排队时延、内存上界和单位经济性，并评审自动扩缩容、重试、熔断和准入等控制回路之间的相互作用。

最低实践：建立 Capacity & Economic Model 和 Control Loop Register。

总纲九：不可逆决策提前设计，可逆决策延迟决定

数据身份、权威边界、外部契约、租户隔离、不可逆写入和安全边界属于高代价决策；内部模块、初始队列大小和局部库通常可以通过实验调整。MVP 应减少不可逆承诺，而不是取消架构纪律。

最低实践：ADR 必须记录假设、替代方案、迁移路径、回滚或 forward-recovery 路径及失效条件。

总纲十：没有证据的关键主张只能是未知

关键性质必须形成：

Claim -> Property -> Mechanism -> Verification -> Evidence -> Gate -> Runtime Feedback

门禁不得把缺失、过期或错绑证据解释为通过；运行漂移应自动降低相关结论的可信度；事故必须推动性质、机制、验证或控制发生变化。

最低实践：维护 Verification & Evidence Matrix，并保护 Assurance Plane 自身。

5. 完整的架构推导模型

推荐的推导顺序是：

```
Business Goal
-> Problem / Goals / Non-goals
-> Constraints / Assumptions / Risk
-> Workload / SLO / Recovery Objectives
-> Scoped Critical Properties / Precedence
-> State / Identity / Ownership
-> Boundary Contracts / Failure Model
-> Consistency / Delivery / Recovery Semantics
-> Architecture / Planes / Control Loops
-> Resource Bounds / Capacity / Economics
-> Contracts / Security / Evolution
-> Verification / Evidence / Gate
-> Runtime Drift / Incident Learning
-> Revised Assumptions / Properties / Architecture
```

这个顺序不是形式主义，而是依赖关系：

- 没有工作负载，就无法合理确定并发模型和容量；
- 没有不变量，就无法决定一致性和事务边界；
- 没有状态权威，就无法定义恢复、去重和冲突裁决；
- 没有失败模型，就无法证明重试和 ACK 顺序安全；
- 没有资源上界，就无法判断系统在持续过载下是否稳定；
- 没有证据绑定，就无法知道门禁结论是否适用于当前部署对象。

架构可以被抽象为受约束优化：

```
maximize:
  business utility

subject to:
  critical properties
  SLO and recovery objectives
  security and compliance boundaries
  cost ceiling
  engineering and operational capacity
  delivery window

minimize where possible:
  accidental complexity
  coordination points
  irreversible commitments
  recovery ambiguity
  epistemic debt
```

这里的“最优”不是单指标极致。吞吐更高但无法恢复、延迟更低但成本不可持续、功能更多但状态语义模糊，都不能称为优秀设计。

6. 从问题、约束与风险开始

6.1 问题定义

高质量问题定义至少包含：

```
Context
Business outcome
Actors and authority
Input / output
Current failure or opportunity
Success criteria
Non-goals
Consequences of failure
Decision deadline
```

“建设一个高性能数据平台”不是工程问题，因为它没有说明价值、负载和失败后果。更好的表述是：

```
对象存储中持续产生不超过 40 MiB 的 CAN 文件；
每个文件约含 3000 个独立 gzip member，元数据由 Kafka 投递；
系统需要按车型与不可变 DBC 版本进行解码，在有界资源内水平扩展；
重复投递、进程崩溃和依赖退化不得产生重复正式结果或错误 DBC 解码；
当前阶段优化目标是持续吞吐和可恢复性，而不是单文件最低尾延迟。
```

6.2 约束与假设的区别

- **约束**：设计必须遵守的现实边界，例如法规、预算、硬件、交付窗口、既有协议；
- **假设**：当前被认为成立、但可能失效的事实，例如对象大小分布、依赖延迟或团队支持能力。

关键假设不得停留在文档备注中。验证结论、生命周期、监测状态和风险标记是独立维度，不构成单一状态链。字段与迁移规则以 [ASSURANCE §6.2](#) 为唯一受控定义：

```
validation_status: UNVALIDATED | VALIDATED | VIOLATED | UNKNOWN
lifecycle_status: PROPOSED | ACTIVE | EXPIRED | RETIRED
monitoring_status: MONITORED | UNMONITORED | UNAVAILABLE | NOT_APPLICABLE
risk_flag: NONE | AT_RISK
```

例如已启用监测的假设仍可能尚未验证；证据过期不等于事实已被违反。替换假设使用新 ID 与 `supersedes` 关系，保留旧记录。

每项关键假设应包含：

字段	含义
Statement	精确、可证伪的陈述
Dependents	哪些性质、ADR、容量或安全结论依赖它
Evidence	当前支持证据与置信理由
Owner	负责监测、复审和失效响应的主体
Signal	运行时观测信号
Expiry	复审日期或失效条件
Response	假设失效后的限流、隔离、复审或迁移动作

无 Owner、无监测、无到期时间的关键假设形成**认识债务 (Epistemic Debt)**。

6.3 工作负载模型

工作负载必须同时描述：

- 到达率、字节率、对象率和业务事件率；
- 平均值、p95、p99、p99.9、硬上限和突发持续时间；
- 读写比例、数据偏斜、租户偏斜和热点；
- CPU、I/O、网络、内存、压缩、序列化和外部调用成本；
- 有序性、重复、乱序、迟到、回放和补算特征；
- 正常、峰值、恢复积压和依赖降容场景；
- 数据增长与未来容量窗口。

只写“峰值 QPS”通常不够。一个系统可能 QPS 不高，但单请求产生大量 fan-out、内存驻留或外部副作用。

6.4 SLO、恢复目标与正确性不得混写

必须区分：

类型	示例
Safety	不得使用未知 DBC 生成正式结果
Liveness	已接纳任务最终完成或进入显式终态
Durability	已提交结果在规定故障模型下不丢失
Availability	每月成功服务时间比例
Latency SLO	99% 文件在 10 分钟内完成
RPO	灾难后最多允许丢失多少已确认状态
RTO	在多长时间恢复到定义能力
Policy	当前优先处理某类车型

SLO 可以被预算和降级；硬安全不变量通常不能被“可用性目标”覆盖。

6.5 风险决定设计深度

设计深度应由三个维度共同决定：

$$\text{Design Depth} \sim \text{Consequence} \times \text{Uncertainty} \times \text{Irreversibility}$$

- 后果低、可逆、可隔离的实验可以快速迭代；
- 涉及权威数据、资金、安全边界或外部长期契约的决策必须更早、更深地建模；
- 不确定性高但可逆的部分优先用原型和观测缩小未知；
- 不确定性高且不可逆的部分应优先降低暴露半径或延迟承诺。

7. 关键性质：从愿望到带作用域的契约

7.1 Safety、Liveness 与其他性质

一个成熟的性质模型至少区分：

- **Safety**：坏事永远不会发生；
- **Liveness**：在明确前提下，好事最终会发生；
- **Durability**：已确认事实在规定故障模型下保持；
- **Isolation / Security**：未授权主体不能观察或修改受保护状态；
- **Recoverability**：系统可从权威状态收敛到一致状态；
- **SLO**：在统计窗口内满足的服务目标；
- **Assumption**：性质成立依赖的外部事实；
- **Policy**：可调整的业务或运营选择。

把所有内容都叫“不变量”会模糊语义。例如“任务最终完成”是活性要求，不是安全不变量；“p99 小于 100ms”是统计 SLO，不应伪装为永真命题。

7.2 属性契约

关键性质至少应表达为：

```
Property = <
  identity,
  scope,
  temporal semantics,
  assumptions,
  guarantee,
  allowed degradation,
  authority boundary,
  verification coverage,
  evidence freshness,
  owner
>
```

示例：

```
Identity:
  source_object_version + pipeline_semantic_version

Scope:
  Result Store 的正式 completion manifest

Temporal semantics:
  在任意可观察时刻最多存在一个 committed manifest

Assumptions:
  source object version 不变; Result Store conditional create 具有线性化语义

Guarantee:
  重复投递、重试或 worker crash 不会形成第二个正式结果

Allowed degradation:
  冲突执行可进入 reconciliation, 但不得覆盖既有正式结果

Verification coverage:
  duplicate delivery、commit/ACK crash、consumer rebalance、worker restart

Evidence freshness:
  仅对当前制品、配置、存储实现和测试故障档案有效
```

这能防止把局部性质夸大成端到端性质，也能暴露性质赖以成立的边界假设。

7.3 性质优先级与冲突

真实系统中的性质可能冲突：完整性要求暂停未知输入，可用性要求继续输出；安全要求 fail closed，应急恢复可能要求 break-glass。系统必须预先定义优先级和裁决机制。

一个常见但非普适的顺序是：

人身安全 / 法律硬约束

- > 权威数据完整性与不可逆业务正确性
- > 身份、授权与租户隔离
- > 可恢复性与证据完整性
- > 核心活性与可用性
- > 延迟和吞吐 SLO
- > 单位成本与局部性能

每个系统都应根据自身风险建立 Property Precedence Matrix。任何事故期间允许牺牲的性质必须提前说明，不能由现场人员临时猜测。

7.4 不变量必须映射到机制

“不重复”“高可用”“数据安全”都不是可执行设计。高质量表达应回答：

什么对象？
在什么作用域？
从什么时候到什么时候？
在什么假设下？
由哪个权威边界维护？
冲突时谁裁决？
如何验证？
违反后如何观测和恢复？

8. 状态、身份、边界与所有权

8.1 先问状态，再问服务

系统边界应从状态和责任推导，而不是先拆服务再分配数据库。对每份重要状态，必须回答：

- 它表示什么业务事实；
- 它的稳定身份是什么；
- 权威副本在哪里；
- 谁能创建、修改、删除和读取；
- 是否不可变、版本化或派生；
- 冲突如何裁决；
- 进程崩溃后从哪里恢复；
- 哪个团队承担 SLO、升级和事故责任。

状态可分为：

类型	说明	设计重点
Authoritative	决定业务事实的权威状态	唯一写入语义、审计、恢复
Durable Derived	可重建但长期保存的派生状态	来源版本、重建成本、漂移
Ephemeral	进程或任务生命周期内状态	丢失后的安全恢复
Evidence	用于证明和门禁的状态	不可静默修改、主体绑定、时效
Configuration / Policy	影响行为或语义的控制状态	版本、权限、发布与回滚

8.2 所有权是多维的

至少区分：

- **Semantic / Technical Owner**：定义状态语义、契约和兼容性；
- **Operational Owner**：承担 SLO、容量、发布、恢复和事故响应；
- **Security Owner**：定义访问、审计和高权限变更边界；
- **Mutation Authority**：真正有权改变权威状态的主体或流程。

“由某服务拥有”不够，因为服务名不能替代组织责任。

8.3 Assume–Guarantee 边界契约

组件局部正确不能自动推出端到端正确。关键边界应表达为：

```
Component C assumes A from its environment.  
If A holds, C guarantees G.  
If A is violated, C produces explicit bounded failure F.
```

例如：

```
Framer assumes:  
  object bytes 与声明 version 一致，且文件不超过支持上限。  
  
Framer guarantees:  
  每个 frame 对应完整 gzip member；损坏边界不会进入正式 decode。  
  
If assumptions fail:  
  返回 typed corruption / unsupported-size，隔离该对象，不猜测继续。
```

端到端主张只有在上游保证真实满足下游假设、所有外部假设均被治理、违反路径有界且可观测时才成立。

8.4 平面划分

成熟系统通常需要区分：

平面	责任
Data Plane	高频数据传输、计算和业务执行
Control Plane	策略、配置、调度、路由和状态收敛
Management Plane	部署、升级、诊断、恢复和人工操作
Assurance Plane	验证、证据、策略门禁、例外和运行漂移判定

分离不是为了增加服务，而是为了防止不同变化速度、权限和失败语义相互污染。

9. 失败、一致性、提交与恢复

9.1 失败是部分进展，而不只是错误码

分布式失败最危险之处在于参与者对“发生了什么”拥有不同视图：

输出已持久化，但 ACK 未提交；
 请求超时，但远端操作可能已经成功；
 配置发布到一半，部分实例已经切换；
 主库提交成功，派生索引尚未更新；
 worker 崩溃，本地状态丢失但外部副作用仍存在。

因此，失败模型必须描述：

- 失败发生在哪个边界和阶段；
- 哪些副作用可能已发生；
- 参与者能观察到什么；
- 哪份状态拥有最终裁决权；
- 重试是否安全；
- 是否需要 reconciliation、补偿或人工裁决；
- 最大积压、恢复时间和数据暴露是什么。

9.2 一致性是推导结果

推荐推导链：

Business Property
 -> Required visibility / ordering / atomicity / isolation
 -> Coordination mechanism
 -> Availability and latency cost
 -> Operational complexity

强一致可能是正确选择，也可能是昂贵的过度协调；最终一致可能足够，也可能直接破坏库存、余额、唯一性或授权边界。应选择**满足关键性质的最弱充分语义**。

9.3 Delivery 与 Effect 分离

必须区分：

- at-most-once delivery;
- at-least-once delivery;
- exactly-once delivery;
- exactly-once business effect。

工程上常用的可靠模式是：

```
at-least-once execution
+ stable operation identity
+ atomic conditional commit / deduplication
+ explicit conflict semantics
= exactly-once business effect within a defined authority boundary
```

这里最后一句的作用域不能省略。它通常只在某个权威存储或业务提交边界成立，而不是证明整个世界“只执行一次”。

9.4 ACK 顺序必须与业务完成语义一致

若消息 ACK 早于权威结果提交，崩溃可能造成任务丢失；若结果提交早于 ACK，重复投递必须由幂等提交吸收。设计必须明确：

```
accepted
started
side effect attempted
authoritative commit succeeded
acknowledged
visible to downstream
```

哪些状态是内部进展，哪个状态才构成业务完成。

9.5 恢复必须从权威状态开始

恢复不得依赖“某个进程应该还记得”。正确恢复通常以持久权威状态为基础，结合：

- create-only 或 conditional update;
- generation、version、fence 或 lease;
- immutable source + replay;
- reconciliation loop;
- compensation 或 forward repair;
- 明确的人工裁决入口。

9.6 显式运行模式

系统应把以下模式建模为一等状态：

```
NORMAL
DEGRADED
RECOVERY
QUARANTINE
MAINTENANCE
EMERGENCY
```

每种模式必须定义：进入条件、授权主体、硬性质、可降级目标、禁止行为、资源策略、最大持续时间、退出条件与证据。降级不是“所有质量都降低”，而是按预先定义的优先级牺牲可牺牲性质，同时保护硬不变量。

10. 有界资源、背压与动态稳定性

10.1 所有增长都必须有上界

必须显式限制：

```
max inflight requests / files / bytes
max queue depth or queued bytes
max worker tasks / threads
max decompression and decode jobs
max open connections / file descriptors
max retry attempts and retry time
max temporary disk
max reconciliation backlog
max cardinality retained in memory
```

上界应优先使用真实稀缺资源单位。例如对象大小差异很大时，inflight bytes 往往比“文件数量”更可靠。

10.2 Queue Contract

每个队列都必须回答：

问题	要求
为什么存在	解耦哪两个速率或调度域
容量单位	items、bytes、CPU cost 或其他
满载行为	阻塞、暂停、拒绝、降级、丢弃或溢写
背压方向	压力如何传播到真正可控的上游
公平性	是否按租户、优先级或成本隔离
终止语义	drain、shutdown、crash 后如何处理
观测	depth、age、wait time、drop、saturation
Owner	谁维护容量和故障策略

Queue 是容量，不是垃圾桶。无界队列只是把失败推迟成更难恢复的失败。

10.3 Deadline、timeout 与 retry budget

局部超时必须服从端到端 deadline。若每层独立重试，最坏请求量可能呈乘法放大。应共享：

```
request deadline
retry budget
attempt identity
backoff and jitter policy
maximum amplification ratio
```

只有瞬态且重试安全的失败才应自动重试；永久错误、权限错误、结构损坏和未知语义通常应快速失败或隔离。

10.4 控制回路稳定性

准入、重试、熔断、自动扩缩容、负载均衡、缓存和人工处置本质上是反馈控制回路。多个局部合理回路叠加后可能形成振荡：

```
依赖变慢
-> timeout 增加
-> retry 放大
-> queue 增长
-> autoscaler 延迟扩容
-> 冷启动争夺依赖
-> 依赖进一步变慢
```

评审至少应检查：

- 传感信号与观测延迟；
- 控制器目标、阈值和时间尺度；
- 执行器能力、动作上界和安全默认；
- hysteresis、cooldown、rate limit；
- 多个控制器是否操纵同一变量；

- 控制器不可用或误判时的失效方式；
- 是否通过仿真、故障注入或历史回放验证稳定性。

10.5 Little's Law 的最低限度应用

稳定系统中：

$$L = \lambda \times W$$

其中 L 是系统内平均在途数量， λ 是完成率， W 是平均停留时间。它提醒设计者：延迟上升会直接增加在途量和内存压力；队列变长不是免费吸收峰值，而是在积累未来必须处理的工作。

11. 容量、单位经济性与复杂度预算

11.1 容量模型必须能够粗算

最小容量模型至少包括：

```
arrival rate
item / byte size distribution
work per item
service time per stage
parallelism and utilization target
memory residency
network and storage bandwidth
recovery backlog and catch-up window
headroom
```

例如：

```
100 files/s x 3000 blocks/file = 300,000 blocks/s
300,000 x 50 microseconds CPU/block = 15 CPU-seconds/second
```

这只给出理论 CPU 下界；实际还必须加入解压、路由、序列化、缓存未命中、调度、抖动和恢复余量。

11.2 Pipeline 吞吐由瓶颈决定

以下速率全部已折算为源输入字节等价吞吐，不是各阶段自身输入或输出字节率：

```
fetch:      10 GB/s
decompress:  4 GB/s
decode:      1 GB/s
output:      8 GB/s
```

理论瓶颈上限为 1 GB/s 源输入等价吞吐；只有阶段独立、负载稳定且调度开销可忽略时，端到端吞吐才可能接近该值。一般按 $\lambda_{\max} \leq \min_i(C_i / d_i)$ 计算： C_i 是资源容量， d_i 是每单位源输入消耗的同单位资源；共享资源、解压膨胀、输出放大与竞争必须另计。不能直接比较压缩字节、解压字节和输出字节的原始 GB/s。继续优化非瓶颈下载不能突破当前上限。性能工程必须区分：

- service time 与 queue time;
- throughput 与 utilization;
- steady state 与 recovery state;
- average 与 tail;
- 单阶段微基准与端到端约束。

11.3 Headroom 是恢复能力的一部分

容量只够平均负载意味着系统无法应对突发、节点故障、升级降容和积压恢复。Headroom 应根据故障域、扩容延迟、恢复窗口和成本目标明确，而不是习惯性保留固定百分比。

11.4 单位经济性

成熟模型应计算：

```
$/request
$/GB processed
$/1M events
$/tenant
$/vehicle-hour
cost of recovery
cost of retained evidence
```

当精确成本数据不可得时，不应放弃，而应标记置信等级：

等级	含义
C0	无成本模型，只能标记 UNKNOWN
C1	依据公开价格或资源配额的粗估
C2	依据账单、标签或节点分摊的工作负载估算
C3	依据实际利用率、共享成本和业务单位的校准模型

平台工程应逐步把 C1/C2 升级为可操作的成本遥测，但不能因为尚未达到 C3 就伪造精确数字。

11.5 复杂度预算

系统复杂度的风险来自可达状态空间、协调点和认知负担，而不只是代码行数。建议跟踪：

- authoritative stores 数量；
- 跨权威边界写入与双写路径；
- lock、lease、fence 和一致性协调点；
- 异步 channel、retry loop 和补偿路径；
- 同时在线的 Schema、协议与配置版本；
- 特权主体、break-glass 和人工恢复步骤；
- 运行模式、quarantine 类型和长期例外；

- 无 Owner、无删除路径或无失效条件的组件。

新增持久状态和分布式边界必须说明必要价值、失败方式、运营责任和未来删除条件。优秀架构不仅会增加机制，也会持续回收复杂度。

12. 契约、安全、演进与可逆性

12.1 契约优先于实现

组件边界必须显式定义：

- identity 与 idempotency key；
- schema、version 与兼容规则；
- ordering、duplication、timeout 和 retry 语义；
- unknown、malformed、not-found 和 conflict 行为；
- authority、ownership 和 mutation rights；
- observability、audit 与 retention。

契约中的错误和未知路径与成功路径同等重要。

12.2 影响语义的内容必须版本化

包括但不限于：

```
schema
protocol
configuration
policy
algorithm
DBC / model / reference data
feature flags with semantic impact
build and dependency graph
```

业务结果应能够追溯到源数据身份、处理语义版本和关键配置版本。

12.3 Replayability

理想的数据处理系统具有：

```
immutable raw input
+ versioned processing logic
+ versioned configuration / reference data
+ deterministic or explainable commit semantics
-> reproducible output
```

重放能力支持审计、修复、迁移、版本比较和事故分析，但其成本、访问权限和结果替换语义必须明确。

12.4 安全从信任边界开始

必须识别：

- 谁能创建任务、修改策略、读取数据和提交结果；
- 哪些输入不可信；
- 哪些控制面操作拥有高爆炸半径；
- 哪个身份代表人、服务、流水线或恢复主体；
- 哪些权限只能通过时限、范围受限的 **break-glass** 获得；
- 谁能够修改验证规则、**required checks** 和豁免逻辑。

安全机制应遵循最小权限、显式身份、默认拒绝、不可静默修改和可审计变更。

12.5 决策可逆性

ADR 除了 Decision 与 Rationale，还必须记录：

```
assumptions
alternatives
consequences
risks
reversibility
migration / rollback / forward-recovery path
invalidation conditions
review triggers
```

对于数据结构和不可逆写入，回滚往往不是切回旧二进制。应优先使用 **expand-migrate-contract**、双读、影子验证、**versioned write**、**reconciliation** 和逐步收缩。

13. 工程保证：从设计主张到持续证据

13.1 保证链

```
Business Requirement
-> Critical Property
  -> Assumption / Boundary Contract
    -> Enforcement Mechanism
      -> Verification
        -> Evidence
          -> Artifact / Config / Environment Identity
            -> Gate Decision
              -> Runtime Observation
                -> Revised Model
```

每个箭头都必须可追踪。断裂的链条意味着只能保持 **UNKNOWN**。

13.2 验证层级

层级	主要能力	不能替代什么
Type / compile checks	表示、接口和局部静态约束	业务语义与运行环境
Static analysis / policy	已编码规则和禁止模式	未编码假设
Unit / property tests	局部行为和广泛输入性质	真实边界组合
Contract / integration tests	边界协议与依赖交互	生产规模和动态稳定性
Load / benchmark	容量、尾延迟、资源上界	故障恢复与业务正确性
Fault injection / chaos	失败路径和控制回路	未注入的未知故障
Formal methods	对精确模型的强推理	模型与现实的一致性
Runtime monitoring	真实运行分布与漂移	未观测、未编码属性
Review / audit	隐含假设、社会技术风险	可重复机器验证

高风险性质通常需要多层互补证据，而不是单一工具。

13.3 Evidence Envelope

关键证据至少应绑定：

```
Evidence {
  evidence_id
  claim_id
  subject_digest
  configuration_digest
  environment_identity
  workload_corpus_digest
  fault_profile
  verifier_identity
  produced_at
  validity / invalidation rule
  coverage
  limitations
  result
  provenance
}
```

以下仅为 ASSURANCE §12.3 的阅读摘要，证据生命周期的受控定义以该处为准：

```
CURRENT
STALE
INVALIDATED
SUPERSEDED
NOT_REPRODUCIBLE
```

环境漂移、关键依赖升级或负载越界应触发结论重评；支持不足时为 UNKNOWN，已有适用反证时保留 CONTRADICTED。不得通过失效处理清除不利事实。

13.4 组合保证

服务 A 和 B 各自测试通过，不能自动推出 $A \rightarrow B$ 端到端性质成立。必须检查：

- Guarantee(A) 是否满足 Assumption(B)；
- 超时、乱序、重复和错误是否也在契约内；
- 容量假设在故障降容时是否仍成立；
- 版本滚动期间的混合组合是否被覆盖；
- 局部证据是否与同一制品、配置和环境组合相匹配。

13.5 Assurance Plane 元不变量

保证面自身也可能失效或被绕过。高风险系统至少应维护：

- MA-01 缺失、过期或错绑证据不得解释为 PASS。
- MA-02 验证规则、门禁策略和豁免逻辑的变更必须经过独立门禁。
- MA-03 高风险变更的 proposer、evaluator 与 final approver 不得由同一非受控主体兼任。
- MA-04 例外和 break-glass 必须有范围、到期时间、审计和补偿复审。
- MA-05 已用于裁决的证据不得被静默修改。
- MA-06 制品、配置、部署目标与运行证据必须通过不可混淆身份关联。
- MA-07 保证面不可用时，高风险变更不得默认放行。
- MA-08 运行漂移超出已验证范围时，相关结论必须自动降级。

13.6 变更是受控实验

重要变更应定义 Change Envelope：

```
semantic delta
assumptions
maximum exposure
blast-radius boundary
observation window
success criteria
abort conditions
rollback / forward-recovery path
owner and authority
evidence to retain
```

Canary 不是“先放少量流量看看”，而是预先定义成功、停止和扩大条件的实验。

13.7 事故必须推动保证体系增长

每次重大事故或近失事件至少应改变以下一项：

Critical Property
Assumption / Boundary Contract
Failure Model
Mechanism
Verification Scenario
Runtime Evidence / Alert
Runbook / Authority
Capacity / Economic Model
Architecture Decision

如果复盘只增加提醒和文字，而没有改变系统属性、控制、验证或权限结构，系统对同类事故的抵抗能力通常没有实质增长。

13.8 防止 Goodhart 化

测试通过率、覆盖率和门禁数量一旦成为目标，团队可能优化指标而非风险。应：

- 从关键性质和故障场景出发选择验证；
- 记录不能证明的内容和覆盖空白；
- 使用负向测试、mutation testing 和未知输入挑战；
- 用真实事故、生产分布和近失事件校准测试集；
- 删除与明确风险无关的低价值门禁；
- 对关键性质保留独立挑战和人工审查。

14. 组织、成熟度与渐进式采用

14.1 Conway 定律不是借口，而是约束

技术边界若没有相应责任边界，最终会在事故和演进中失效。系统设计应同时优化：

state authority
bounded context
team ownership
on-call and recovery path
change coordination cost

平台团队不应通过中央审批拥有所有变更，而应提供有约束的自助能力、默认安全路径和可审计例外。

14.2 渐进式采用

不同风险等级需要不同厚度：

类型	最低要求
探索型原型	Non-goals、数据安全边界、不可逆决策、退出条件
有限业务系统	关键性质、状态权威、失败路径、资源上界、基础证据
核心生产系统	完整门禁、容量与恢复模型、运行模式、证据绑定
高风险/合规系统	独立审批、组合保证、保证面元不变量、例外治理与持续漂移检测

“渐进”意味着保证深度与风险匹配，不意味着关键数据身份和安全边界可以缺失。

14.3 成熟度模型

等级	特征
L0 — Component-driven	以技术栈和拓扑为中心
L1 — Functional	正常路径可用，但失败、状态和责任模糊
L2 — Bounded	资源有界，具备基本失败与恢复语义
L3 — Operable	SLO、Owner、发布、回滚、容量和可诊断性明确
L4 — Evidence-driven	关键主张映射到验证、证据和门禁
L5 — Adaptive Assurance	假设、证据和环境可关联；漂移自动失效结论；事故驱动保证增量

成熟度不能通过采购更多基础设施跳级。缺少状态语义和所有权时，引入更多组件通常只会扩大状态空间。

15. 最终判定标准

一个系统设计只有在能够回答以下问题时，才具备进入生产评审的基础：

1. 系统解决什么问题，明确不解决什么？
2. 真实工作负载、风险、约束和关键假设是什么？
3. 哪些安全性质、活性要求和恢复目标必须成立？
4. 每份权威状态和稳定身份由谁拥有、修改与恢复？
5. 边界双方的假设、保证和违反行为是什么？
6. 重复、超时、乱序、部分成功和资源耗尽时如何收敛？
7. 一致性、ACK、幂等和冲突语义如何维护业务性质？
8. 所有可增长资源是否有界，压力能否端到端传播？
9. 峰值、故障降容、积压恢复和单位成本是否经过计算？
10. 正常、降级、恢复和紧急模式分别允许什么？
11. 系统如何升级、迁移、回滚、重放、排空和灾难恢复？
12. 每项关键主张的机制、验证、证据、覆盖和失效条件是什么？
13. 保证面如何防止被绕过、弱化或错误地默认放行？

14. 假设或环境失效后，谁按 ASSURANCE 重评结论、保留反证与未知并触发复审？
15. 系统新增的复杂度何时、由谁、在什么条件下回收？

结论

高级系统工程并不是熟悉更多组件，而是能够维持一条完整、诚实且可复审的推理链：为什么存在，什么不能错，什么可能错，谁拥有事实，需要什么语义，资源和成本是多少，如何恢复与演进，以及我们凭什么相信这些结论仍然适用于当前系统。

附录 A：核心术语边界

术语	严格含义
Critical Property	对业务、正确性、安全、活性、资源或运营具有关键影响，并具有与性质类型匹配的验证义务的上位概念
Invariant	在规定身份、作用域和允许状态转换内始终成立的 Safety 性质，不应泛指所有目标
Safety	“坏事永远不发生”的性质，例如不重复正式提交、不越权写入
Liveness	在明确假设下，“好事最终发生或进入明确终态”的性质
Durability	在指定故障集合内，已确认事实保持可恢复的性质
SLO	在统计窗口和作用域内对服务结果的目标，不等同于正确性硬约束
Assumption	某项性质、容量或组件保证赖以成立、可被证伪并可能到期的外部条件
Guarantee	在假设成立时，组件、边界或系统承诺提供的行为
Authority Boundary	对某项业务事实拥有最终裁决权的状态或组件边界
Engineering Assurance	从主张、已编码性质、机制、验证、证据、门禁到运行反馈的持续控制过程
Evidence Envelope	证据所绑定的制品、配置、环境、工作负载、故障模型、覆盖、限制与有效期
Epistemic Debt	关键假设未验证、无 Owner、无监测或无失效响应所形成的认识风险
Change Envelope	对变更语义、最大暴露、观察窗口、成功/停止条件和恢复路径的预先约束
Assurance Delta	事故或运行事实对性质、假设、机制、验证、证据或治理产生的可审计增量

附录 B：参考资料

以下资料提供分布式系统、可靠性、软件复杂度、安全工程、控制与验证方面的理论背景；它们不替代具体项目的约束分析和实证证据：

1. Martin Kleppmann, *Designing Data-Intensive Applications*.
2. Michael T. Nygard, *Release It!*.
3. Betsy Beyer et al., *Site Reliability Engineering*.
4. Eric Evans, *Domain-Driven Design*.
5. John Ousterhout, *A Philosophy of Software Design*.

6. Nancy Leveson, *Engineering a Safer World*.
7. Leslie Lamport, “Time, Clocks, and the Ordering of Events in a Distributed System”.
8. Leslie Lamport, *Specifying Systems*, 尤其第 8 章 Liveness and Fairness、第 9 章 Real Time 与第 14 章模型检查。
9. Pat Helland, “Life Beyond Distributed Transactions: An Apostate's Opinion”.
10. Gerard J. Holzmann, *The SPIN Model Checker*.
11. John D. C. Little, “A Proof for the Queuing Formula: $L = \lambda W$ ”.
12. RFC 2119 and RFC 8174, requirements language for normative specifications.

附录 C：历史 v1.1.0 关键校准

以下记录保留 v1.1.0 相对早期 v1.0 稿件的历史变化，不代表当前草案已获发布批准：

1. 用 Critical Property 区分并统摄 Safety、Liveness、Durability、Resource Bound 与 SLO，不再把所有目标泛称为 invariant；
2. 把关键性质升级为带 identity、scope、temporal semantics、assumptions、allowed degradation 和 evidence freshness 的契约；
3. 增加 Property Precedence，提前裁决完整性、可用性、安全和恢复之间的冲突；
4. 增加 Assume–Guarantee 边界契约，防止局部通过被夸大为端到端保证；
5. 把假设、证据时效、运行漂移和 UNKNOWN 纳入正式治理；
6. 把显式运行模式、控制回路稳定性、单位经济性和复杂度预算纳入架构；
7. 为 Assurance Plane 增加元不变量、职责分离和信任链；
8. 把 canary、重大发布和迁移定义为受控实验；
9. 要求重大事故产生可验证的 Assurance Delta；
10. 将方法论与操作手册拆分，避免认知基线和项目模板形成两份互相漂移的重复文档。